

LAPORAN TUGAS 4

Mata Kuliah Sistem Operasi



Di Susun Oleh :

Muhammad Rafi Amiruddin

Nim (2410651027)

Jurusan Teknik Informatika

Fakultas Teknik

Universitas Muhammadiyah Jember

06 November 2025

DAFTAR ISI

LAPORAN TUGAS 4	1
DAFTAR ISI.....	i
1 ISI LAPORAN.....	1
1.1 Tugas 1.....	1
1.1.1 Eksperimen dengan Data Berbeda	1
1.2 Tugas 2.....	4
1.2.1 Analisis Eksperimen ketiga algoritma	4
1.3 Tugas 3.....	5
1.3.1 Modifikasi Program	5
1.4 Tugas 4.....	9
1.4.1 Eksperimen dengan data berbeda untuk perbandingan algoritma	9
1.5 Tugas 5.....	16
1.5.1 Analisis Eksperimen perbandingan algoritma	16

1 ISI LAPORAN

1.1 Tugas 1

1.1.1 Eksperimen dengan Data Berbeda

- Jalankan setiap algoritma dengan data berikut dan catat hasilnya:

Data Set 1:

P1: AT=0, BT=10

P2: AT=1, BT=5

P3: AT=2, BT=8

Data Set 2:

P1: AT=0, BT=5

P2: AT=1, BT=3

P3: AT=2, BT=8

P4: AT=3, BT=6

- Buat File nano tugas1pertemuan4.c

Yang didalam file berisi syntax :

```
#include <stdio.h>
```

```
struct Process {  
    int pid;  
    int at; // Arrival Time  
    int bt; // Burst Time  
    int ct; // Completion Time  
    int tat; // Turnaround Time  
    int wt; // Waiting Time  
};
```

```
// Fungsi untuk menampilkan hasil
```

```
void printResult(struct Process p[], int n) {  
    float avg_tat = 0, avg_wt = 0;  
    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");  
    for (int i = 0; i < n; i++) {  
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", p[i].pid, p[i].at, p[i].bt,  
p[i].ct, p[i].tat, p[i].wt);  
        avg_tat += p[i].tat;  
        avg_wt += p[i].wt;  
    }  
    printf("\nRata-rata Turnaround Time = %.2f", avg_tat / n);  
    printf("\nRata-rata Waiting Time    = %.2f\n", avg_wt / n);  
}
```

```
// Algoritma FCFS (First Come First Serve)
```

```
void FCFS(struct Process p[], int n) {
```

```

printf("\n=== Algoritma FCFS ===\n");
int time = 0;
for (int i = 0; i < n; i++) {
    if (time < p[i].at) time = p[i].at; // tunggu jika proses belum datang
    time += p[i].bt;
    p[i].ct = time;
    p[i].tat = p[i].ct - p[i].at;
    p[i].wt = p[i].tat - p[i].bt;
}
printResult(p, n);
}

int main() {
    // ===== Dataset 1 =====
    struct Process data1[] = {
        {1, 0, 10},
        {2, 1, 5},
        {3, 2, 8}
    };
    int n1 = 3;

    // ===== Dataset 2 =====
    struct Process data2[] = {
        {1, 0, 5},
        {2, 1, 3},
        {3, 2, 8},
        {4, 3, 6}
    };
    int n2 = 4;

    printf("DATA SET 1:");
    FCFS(data1, n1);

    printf("\n\nDATA SET 2:");
    FCFS(data2, n2);

    return 0;
}

```

- Lalu Compile dan jalankan dengan `gcc tugas1pertemuan4.c -o tugas1pertemuan4` dan `./tugas1pertemuan4`

Ini adalah Gambaran hasil tugas1pertemuan4

Program ini dibuat buat ngetes dan ngebandingin hasil penjadwalan CPU pakai algoritma FCFS (First Come First Serve) pada dua kumpulan data (Data Set 1 dan Data Set 2). FCFS itu artinya proses yang datang paling duluan bakal dijalankan paling dulu kayak sistem antrian biasa.

Program ini nunjukin gimana FCFS bekerja secara berurutan sesuai kedatangan proses.

- Proses pertama selalu langsung jalan.
- Proses yang datang belakangan harus nunggu proses sebelumnya selesai, jadi makin panjang burst time proses pertama, makin lama proses lain nunggu.

Karena itu, FCFS bisa dibilang sederhana tapi kurang efisien kalau ada proses dengan waktu eksekusi yang panjang di awal (bikin yang lain nunggu lama).

```
rafi@satelit:~$ nano tugas1pertemuan4
rafi@satelit:~$ nano tugas1pertemuan4.c
rafi@satelit:~$ gcc tugas1pertemuan4.c -o tugas1pertemuan4
rafi@satelit:~$ ./tugas1pertemuan4
DATA SET 1:
=== Algoritma FCFS ===

PID    AT    BT    CT    TAT    WT
P1      0     10    10     10     0
P2      1      5    15     14     9
P3      2      8    23     21    13

Rata-rata Turnaround Time = 15.00
Rata-rata Waiting Time    = 7.33

DATA SET 2:
=== Algoritma FCFS ===

PID    AT    BT    CT    TAT    WT
P1      0      5     5     5     0
P2      1      3     8     7     4
P3      2      8    16    14     6
P4      3      6    22    19    13

Rata-rata Turnaround Time = 11.25
Rata-rata Waiting Time    = 5.75
```

Artinya di dataset pertama, rata-rata tiap proses butuh 15 waktu buat selesai sejak datang, dan rata-rata nunggu 7,33 waktu sebelum mulai dijalankan.

Sedangkan di dataset kedua, waktu tunggu dan total penyelesaian lebih kecil dibanding dataset pertama. Ini nunjukin bahwa urutan dan waktu datang proses sangat ngaruh terhadap performa FCFS.

1.2 Tugas 2

1.2.1 Analisis Eksperimen ketiga algoritma

- Jawab pertanyaan berikut:

1. Algoritma mana yang memberikan Average Waiting Time terkecil?
2. Mengapa hasil algoritma berbeda-beda?
3. Kapan sebaiknya menggunakan algoritma FCFS?
4. Kapan sebaiknya menggunakan algoritma SJF?
5. Kapan sebaiknya menggunakan algoritma Round Robin?

- Jawaban :

1. Biasanya SJF memberikan Average Waiting Time (AWT) terkecil.
2. Karena strategi penjadwalan berbeda (urutan kedatangan, durasi terpendek, atau pembagian quantum).
3. Sistem batch dengan durasi proses mirip, ketika kesederhanaan diutamakan.
4. Ketika burst time dapat diprediksi, dan tujuan meminimalkan AWT.
5. Sistem interaktif / multitasking, butuh respon cepat dan fairness.

1.3 Tugas 3

1.3.1 Modifikasi Program

- Tambahkan fitur berikut pada salah satu program:

1. Tampilkan Gantt Chart yang lebih detail
2. Simpan hasil ke file text
3. Baca input dari file

- Buat File nano tugas3pertemuan4.c

File tersebut berisi syntax :

```
#include <stdio.h>
```

```
struct Process {  
    int pid; // Process ID  
    int at;  // Arrival Time  
    int bt;  // Burst Time  
    int ct;  // Completion Time  
    int tat; // Turnaround Time  
    int wt;  // Waiting Time  
};
```

```
// Fungsi untuk menampilkan hasil dan juga menyimpan ke file
```

```
void printAndSaveResult(struct Process p[], int n) {  
    FILE *f = fopen("hasil_fcfs.txt", "w");  
    if (f == NULL) {  
        printf("Gagal membuka file!\n");  
        return;  
    }  
}
```

```
float avg_tat = 0, avg_wt = 0;
```

```
printf("\n\n=== HASIL PENJADWALAN FCFS ===\n");  
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");  
fprintf(f, "PID\tAT\tBT\tCT\tTAT\tWT\n");
```

```
for (int i = 0; i < n; i++) {  
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",  
        p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);  
    fprintf(f, "P%d\t%d\t%d\t%d\t%d\t%d\n",  
        p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);  
  
    avg_tat += p[i].tat;  
    avg_wt += p[i].wt;  
}
```

```

    avg_tat /= n;
    avg_wt /= n;

    printf("\nRata-rata Turnaround Time = %.2f", avg_tat);
    printf("\nRata-rata Waiting Time = %.2f\n", avg_wt);

    fprintf(f, "\nRata-rata Turnaround Time = %.2f", avg_tat);
    fprintf(f, "\nRata-rata Waiting Time = %.2f\n", avg_wt);

    fclose(f);
    printf("\n>> Hasil juga disimpan di file: hasil_fcfs.txt\n");
}

// Fungsi untuk menampilkan Gantt Chart
void printGanttChart(struct Process p[], int n) {
    printf("\n\n=== GANTT CHART ===\n");
    printf(" ");

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < p[i].bt; j++)
            printf("--");
        printf(" ");
    }

    printf("\n|");

    for (int i = 0; i < n; i++) {
        int mid = p[i].bt;
        for (int j = 0; j < mid - 1; j++)
            printf(" ");
        printf("P%d", p[i].pid);
        for (int j = 0; j < mid - 1; j++)
            printf(" ");
        printf("|");
    }

    printf("\n ");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < p[i].bt; j++)
            printf("--");
        printf(" ");
    }

    printf("\n0");
    for (int i = 0; i < n; i++) {
        printf("%*d", p[i].bt * 2, p[i].ct);
    }
    printf("\n");
}

```



```

// Algoritma FCFS
void FCFS(struct Process p[], int n) {
    int time = 0;
    for (int i = 0; i < n; i++) {
        if (time < p[i].at) time = p[i].at;
        time += p[i].bt;
        p[i].ct = time;
        p[i].tat = p[i].ct - p[i].at;
        p[i].wt = p[i].tat - p[i].bt;
    }

    printAndSaveResult(p, n);
    printGanttChart(p, n);
}

int main() {
    // Baca dari input.txt (format: pid at bt per baris)
    struct Process p[20];
    int n = 0;

    FILE *f = fopen("input.txt", "r");
    if (f == NULL) {
        printf("File input.txt tidak ditemukan! Menggunakan data
default.\n");
        struct Process p_default[] = {
            {1,0,10}, {2,1,5}, {3,2,8}
        };
        n = 3;
        for (int i=0;i<n;i++) p[i] = p_default[i];
    } else {
        while (fscanf(f, "%d %d %d", &p[n].pid, &p[n].at, &p[n].bt) == 3)
            n++;
        fclose(f);
    }

    FCFS(p, n);

    return 0;
}

```

- Lalu Compile dan jalankan dengan `gcc tugas3pertemuan4.c -o tugas3pertemuan4` dan `./tugas3pertemuan4`

Ini adalah Gambaran hasil tugas3pertemuan4

Program ini nyimulasikan cara CPU ngerjain proses satu per satu sesuai urutan datangnya (arrival time) yang duluan datang, itu yang dikerjain dulu. Nama programnya tugas3pertemuan4.c, dan hasil output-nya ditampilin di terminal plus disimpan ke file hasil_fcfs.txt.

Program ini ngajarin konsep dasar gimana komputer (CPU) ngatur antrian tugas atau proses yang mau dijalankan. Semua proses bakal dieksekusi sesuai urutan datangnya, kayak antrian orang di loket yang datang duluan dilayani duluan, gak peduli tugasnya lama atau cepat.

```

rafi@satelit:~$ nano tugas3pertemuan4.c
rafi@satelit:~$ gcc tugas3pertemuan4.c -o tugas3pertemuan4
rafi@satelit:~$ ./tugas3pertemuan4
File input.txt tidak ditemukan! Menggunakan data default.

=== HASIL PENJADWALAN FCFS ===
PID    AT      BT      CT      TAT      WT
P1      0       10      10       10       0
P2      1        5      15       14       9
P3      2        8      23       21      13

Rata-rata Turnaround Time = 15.00
Rata-rata Waiting Time    = 7.33

>> Hasil juga disimpan di file: hasil_fcfs.txt

=== GANTT CHART ===
-----
|           P1           |           P2           |           P3           |
-----
0                     10                     15                     23

```

File input.txt tidak ditemukan! Menggunakan data default.

Artinya program nyoba baca data proses dari file input.txt, tapi karena file-nya gak ada, dia pake data bawaan (default) yang udah ditulis langsung di kode.

1.4 Tugas 4

1.4.1 Eksperimen dengan data berbeda untuk perbandingan algoritma

- Test Case 1: Proses dengan burst time bervariasi

Jumlah proses: 4

Time quantum: 3

- P1: AT=0, BT=24

- P2: AT=1, BT=3

- P3: AT=2, BT=3

- P4: AT=3, BT=6

- Test Case 2: Proses datang bersamaan

Jumlah proses: 5

Time quantum: 4

- P1: AT=0, BT=10

- P2: AT=0, BT=5

- P3: AT=0, BT=8

- P4: AT=0, BT=12

- P5: AT=0, BT=6

- Test Case 3: Proses datang dengan interval

Jumlah proses: 5

Time quantum: 2

- P1: AT=0, BT=8

- P2: AT=2, BT=4

- P3: AT=4, BT=9

- P4: AT=6, BT=5

- P5: AT=8, BT=3

- Buat File nano tugas4pertemuan4.c

Yang didalam file tersebut berisi Syntax :

```
#include <stdio.h>
```

```
struct Process {  
    int pid;  
    int at;  
    int bt;  
    int ct;
```

```

    int tat;
    int wt;
    int rt; // remaining time (untuk RR)
};

// Cetak hasil
void printResult(struct Process p[], int n, char *algoname) {
    float avg_tat = 0, avg_wt = 0;
    printf("\n=== %s ===\n", algoname);
    printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        p[i].tat = p[i].ct - p[i].at;
        p[i].wt = p[i].tat - p[i].bt;
        avg_tat += p[i].tat;
        avg_wt += p[i].wt;
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
    }
    printf("Rata-rata TAT = %.2f\n", avg_tat / n);
    printf("Rata-rata WT = %.2f\n", avg_wt / n);
}

// FCFS
void FCFS(struct Process p[], int n) {
    int time = 0;
    for (int i = 0; i < n; i++) {
        if (time < p[i].at) time = p[i].at;
        time += p[i].bt;
        p[i].ct = time;
    }
    printResult(p, n, "FCFS");
}

// SJF Non-preemptive
void SJF(struct Process p[], int n) {
    int completed = 0, time = 0;
    int done[10] = {0};

    while (completed < n) {
        int idx = -1, min_bt = 9999;
        for (int i = 0; i < n; i++) {
            if (!done[i] && p[i].at <= time && p[i].bt < min_bt) {
                min_bt = p[i].bt;
                idx = i;
            }
        }

        if (idx == -1) {
            time++;
            continue;
        }
    }
}

```

```

    }

    time += p[idx].bt;
    p[idx].ct = time;
    done[idx] = 1;
    completed++;
}
printResult(p, n, "SJF Non-preemptive");
}

// Round Robin
void RR(struct Process p[], int n, int tq) {
    int completed = 0;
    int time = 0;

    for (int i = 0; i < n; i++)
        p[i].rt = p[i].bt; // remaining time

    while (completed < n) {
        int all_done = 1;
        for (int i = 0; i < n; i++) {
            if (p[i].rt > 0 && p[i].at <= time) {
                all_done = 0;
                if (p[i].rt > tq) {
                    time += tq;
                    p[i].rt -= tq;
                } else {
                    time += p[i].rt;
                    p[i].rt = 0;
                    p[i].ct = time;
                    completed++;
                }
            } else if (p[i].rt > 0 && p[i].at > time) {
                time++; // tunggu proses berikutnya datang
            }
        }
        if (all_done) break;
    }
    printResult(p, n, "Round Robin");
}

// Salin array agar tidak tercampur antar algoritma
void copyData(struct Process src[], struct Process dest[], int n) {
    for (int i = 0; i < n; i++)
        dest[i] = src[i];
}

// Jalankan satu test case
void runTestCase(struct Process data[], int n, int tq, char *judul) {

```

```

printf("\n\n=====\\n%s\\n=====\\n", judul);

    struct Process fcfs[10], sjf[10], rr[10];
    copyData(data, fcfs, n);
    copyData(data, sjf, n);
    copyData(data, rr, n);

    FCFS(fcfs, n);
    SJF(sjf, n);
    RR(rr, n, tq);
}

int main() {
    // TEST CASE 1
    struct Process tc1[] = {
        {1, 0, 24}, {2, 1, 3}, {3, 2, 3}, {4, 3, 6}
    };
    runTestCase(tc1, 4, 3, "TEST CASE 1: Burst Time Bervariasi");

    // TEST CASE 2
    struct Process tc2[] = {
        {1, 0, 10}, {2, 0, 5}, {3, 0, 8}, {4, 0, 12}, {5, 0, 6}
    };
    runTestCase(tc2, 5, 4, "TEST CASE 2: Semua Proses Datang Bersamaan");

    // TEST CASE 3
    struct Process tc3[] = {
        {1, 0, 8}, {2, 2, 4}, {3, 4, 9}, {4, 6, 5}, {5, 8, 3}
    };
    runTestCase(tc3, 5, 2, "TEST CASE 3: Proses Datang Secara Interval");

    return 0;
}

```

- Lalu Compile dan jalankan dengan `gcc tugas3pertemuan4.c -o tugas3pertemuan4` dan `./tugas3pertemuan4`

Ini adalah Gambaran hasil tugas3pertemuan4

Pada Test Case 1 adalah Burst Time Bervariasi

Artinya tiap proses punya waktu eksekusi (Burst Time) yang beda-beda, dan datangnya nggak barengan (lihat kolom AT = Arrival Time).

- FCFS & SJF hasilnya sama karena urutan datangnya proses sama dengan urutan burst time-nya (jadi nggak ada yang lebih pendek datang duluan).
- Round Robin ngasih hasil TAT (Turnaround Time) & WT (Waiting Time) lebih kecil, karena tiap proses dikasih jatah waktu (quantum) bergiliran, jadi semua kebagian CPU lebih adil.

Intinya: di sini Round Robin lebih efisien karena beban CPU terbagi rata.

```
rafi@satelit:~$ nano tugas4pertemuan4.c
rafi@satelit:~$ gcc tugas4pertemuan4.c -o tugas4pertemuan4
rafi@satelit:~$ ./tugas4pertemuan4
```

```
=====
TEST CASE 1: Burst Time Bervariasi
=====
```

```
=== FCFS ===
```

PID	AT	BT	CT	TAT	WT
P1	0	24	24	24	0
P2	1	3	27	26	23
P3	2	3	30	28	25
P4	3	6	36	33	27

Rata-rata TAT = 27.75
Rata-rata WT = 18.75

```
=== SJF Non-preemptive ===
```

PID	AT	BT	CT	TAT	WT
P1	0	24	24	24	0
P2	1	3	27	26	23
P3	2	3	30	28	25
P4	3	6	36	33	27

Rata-rata TAT = 27.75
Rata-rata WT = 18.75

```
=== Round Robin ===
```

PID	AT	BT	CT	TAT	WT
P1	0	24	36	36	12
P2	1	3	6	5	2
P3	2	3	9	7	4
P4	3	6	18	15	9

Rata-rata TAT = 15.75
Rata-rata WT = 6.75

Namun Pada Test Case 2 Semua Proses Datang Bersamaan

Semua proses mulai di waktu 0 (AT = 0).

- FCFS:urut sesuai antrean masuk (P1, P2, P3, dst). Akibatnya yang di belakang (misal P5) nunggu lama banget.
- SJF Non-preemptive: lebih efisien karena proses dengan waktu singkat dikerjakan dulu. Jadi TAT dan WT rata-rata turun.
- Round Robin: karena semua datang barengan, RR jadi kurang efisien banyak context switching dan semua bagian giliran pendek, jadi total waktunya malah naik.

Intinya: kalau semua proses datang bareng, SJF paling bagus, karena lebih cepat menyelesaikan proses pendek dulu.

```
rafi@satelit: ~  
=====br/>TEST CASE 2: Semua Proses Datang Bersamaanbr/>=====br/>  
=== FCFS ===  
PID    AT    BT    CT    TAT    WT  
P1      0    10    10    10     0  
P2      0     5    15    15    10  
P3      0     8    23    23    15  
P4      0    12    35    35    23  
P5      0     6    41    41    35  
Rata-rata TAT = 24.80  
Rata-rata WT  = 16.60  
  
=== SJF Non-preemptive ===  
PID    AT    BT    CT    TAT    WT  
P1      0    10    29    29    19  
P2      0     5     5     5     0  
P3      0     8    19    19    11  
P4      0    12    41    41    29  
P5      0     6    11    11     5  
Rata-rata TAT = 21.00  
Rata-rata WT  = 12.80  
  
=== Round Robin ===  
PID    AT    BT    CT    TAT    WT  
P1      0    10    37    37    27  
P2      0     5    25    25    20  
P3      0     8    29    29    21  
P4      0    12    41    41    29  
P5      0     6    35    35    29  
Rata-rata TAT = 33.40  
Rata-rata WT  = 25.20
```


Dan pada Test Case 3 yaitu Proses Datang Secara Interval

Proses datang satu-satu dalam waktu berbeda (berjarak).

- FCFS: proses yang datang duluan langsung dikerjakan tanpa mikir mana yang lebih cepat, jadi kadang yang kecil nunggu lama.
- SJF Non-preemptive: lebih pintar karena milih proses yang paling cepat di antara yang sudah datang, jadi TAT dan WT turun.
- Round Robin: hasilnya lumayan bagus juga, tapi karena ada pergantian proses terus (time slice), total waktu kadang lebih tinggi.

Intinya: kalau proses datang nyicil, SJF Non-preemptive tetap paling efisien buat ngurangin waktu tunggu.

```
=====
TEST CASE 3: Proses Datang Secara Interval
=====

=== FCFS ===
PID    AT    BT    CT    TAT    WT
P1      0     8     8     8     0
P2      2     4    12    10     6
P3      4     9    21    17     8
P4      6     5    26    20    15
P5      8     3    29    21    18
Rata-rata TAT = 15.20
Rata-rata WT  = 9.40

=== SJF Non-preemptive ===
PID    AT    BT    CT    TAT    WT
P1      0     8     8     8     0
P2      2     4    15    13     9
P3      4     9    29    25    16
P4      6     5    20    14     9
P5      8     3    11     3     0
Rata-rata TAT = 12.60
Rata-rata WT  = 6.80

=== Round Robin ===
PID    AT    BT    CT    TAT    WT
P1      0     8    26    26    18
P2      2     4    14    12     8
P3      4     9    29    25    16
P4      6     5    24    18    13
P5      8     3    19    11     8
Rata-rata TAT = 18.40
Rata-rata WT  = 12.60
```

1.5 Tugas 5

1.5.1 Analisis Eksperimen perbandingan algoritma

1. Analisis Performa

- Dalam kondisi apa FCFS memberikan hasil terbaik?
- Mengapa SJF hampir selalu memberikan Average WT terkecil?
- Apa trade-off dari Round Robin?

2. Context Switching

- Algoritma mana yang paling banyak melakukan context switch?
- Bagaimana pengaruh quantum terhadap jumlah context switch di Round Robin?
- Coba jalankan Round Robin dengan quantum 2, 4, 8, 16 - apa yang terjadi?

3. Fairness

- Algoritma mana yang paling "adil" untuk semua proses?
- Proses mana yang paling dirugikan di algoritma FCFS?
- Apakah ada kemungkinan starvation di SJF?

4. Real World Application

- Untuk web server yang melayani banyak request kecil, algoritma mana yang cocok?
- Untuk render video (task besar), algoritma mana yang cocok?
- Untuk OS desktop (interactive), algoritma mana yang cocok?

Jawaban :

Setelah menjalankan program dengan berbagai test case, jawab pertanyaan berikut:

1. Analisis Performa

- FCFS terbaik ketika proses memiliki burst time mirip dan sistem batch.
- SJF hampir selalu memberikan Average WT terkecil karena memilih job terpendek lebih dulu (optimal untuk AWT jika durasi diketahui).
- Trade-off RR: adil dan responsif tetapi banyak context switch jika quantum kecil.

2. Context Switching

- RR paling banyak melakukan context switch.
- Quantum kecil -> banyak context switch; quantum besar -> sedikit context switch.

3. Fairness

- RR paling adil.
- FCFS merugikan proses pendek jika ada proses panjang di depan (convoy effect).
- SJF berisiko starvation untuk proses panjang.

4. Real World Application

- Web server (many small requests): SJF atau RR dengan quantum kecil
- Render video (task besar): FCFS atau SJF.
- OS desktop (interactive): Round Robin.